

chapter10 인터넷 검색 및 멀티미디어 통합형

지능형 챗봇 구축

1. 서론: 대규모 언어 모델의 지식 한계와 검색 증강의 필연성

1.1 지식 단절(Knowledge Cutoff) 문제의 본질

모든 LLM이 마주한 벽: 지식 단절 (Knowledge Cutoff)

정의 LLM은 학습이 종료된 시점까지의 데이터만 기억하는 '정적 지식(Static Knowledge)'에 의존합니다. 2023년에 학습된 모델은 2024년의 사건을 알지 못합니다.

원인 모델 내부 파라미터에 저장된 지식을 사용하며, 실시간으로 외부 정보를 습득하지 못하기 때문입니다.

결과

- 신뢰성 하락: 최신 뉴스, 금융 정보 등 시의성이 중요한 질문에 부정확한 답변을 제공합니다.
- 환각(Hallucination) 현상: 모르는 정보에 대해 그럴듯한 거짓 정보를 생성할 위험이 높아집니다.



아키텍처 다이어그램 v1

LLM Core

현대 인공지능 기술의 정점에 있는 대규모 언어 모델(Large Language Model, LLM)은 방대한 텍스트 데이터를 학습하여 인간 수준의 언어 이해 및 생성 능력을 보여줍니다. 그러나 이러한 모델들은 근본적인 구조적 한계를 지니고 있는데, 바로 '지식 단절(Knowledge Cutoff)' 문제입니다. LLM은 학습이 종료된 시점까지의 데이터만을 기억하고 있으며, 그 이후에 발생한 사건이나 생성된 정보에 대해서는 인지하지 못합니다. 예를 들어, 2023년까지의 데이터로 학습된 GPT-4 모델에게 "2024년 미국 대선 결과는 어떻게 되었는가?"라고 질문한다면, 모델은 정확한 답변을 제공할 수 없습니다. 이는 모델이 실시간으로 인터넷에 접속하여 정보를 습득하는 것이 아니라, 모델 내부의 파라미터(Parameter)에 저장된 정적 지식(Static Knowledge)에 의존하여 답변을 생성하기 때문입니다.

이러한 한계는 특히 급변하는 뉴스, 금융 시장 정보, 최신 기술 트렌드 등을 다루는 애플리케이션에서 치명적인 약점으로 작용합니다. 사용자는 항상 최신 정보를 원하며, 과거의

데이터에 머물러 있는 챗봇은 신뢰성을 잃게 됩니다. 또한, 모델이 자신이 모르는 정보를 질문받았을 때, 그럴듯해 보이지만 사실이 아닌 정보를 생성해내는 '환각(Hallucination)' 현상이 발생할 위험도 높아집니다. 이는 챗봇 서비스의 품질을 저하시키고 사용자 경험을 해치는 주요 원인이 됩니다.

1.2 검색 증강 생성(RAG)과 에이전트의 부상



이러한 문제를 해결하기 위해 등장한 개념이 바로 검색 증강 생성(Retrieval-Augmented Generation, RAG)입니다. RAG는 LLM이 답변을 생성하기 전에 외부의 지식 베이스(Knowledge Base)나 인터넷 검색 엔진을 통해 관련 정보를 실시간으로 검색(Retrieve)하고, 검색된 정보를 프롬프트(Prompt)에 문맥(Context)으로 포함시켜 답변을 생성(Generate)하는 기술입니다. 이를 통해 LLM은 재학습(Fine-tuning) 없이도 최신 정보를 반영한 답변을 제공할 수 있게 됩니다.

랭체인(LangChain)과 같은 오케스트레이션 프레임워크는 이러한 RAG 시스템을 더욱 고도화하여 '에이전트(Agent)'라는 개념을 도입했습니다. 에이전트는 단순히 검색 결과를 받아쓰는 것을 넘어, 사용자의 질문 의도를 파악하고, 스스로 어떤 도구(Tool)를 사용할지 판단하며, 필요한 경우 웹 검색, 데이터베이스 조회, 계산기 실행 등 다양한 행동을 수행하는 자율적인 시스템입니다. 이러한 에이전트 기술을 활용하여, 텍스트 기반의 웹 검색뿐만 아니라 유튜브와 같은 멀티미디어 데이터까지 실시간으로 처리하는 차세대 챗봇 구축 방법론을 상세히 분석합니다.

2. 오픈소스 기반 웹 검색 구현: DuckDuckGo 와 정밀 스크래핑

2.1 검색 엔진의 선택: DuckDuckGo 의 기술적, 윤리적 이점

인터넷 검색 기능을 챗봇에 통합하기 위해서는 검색 엔진 API 가 필수적입니다. Google Search API 나 Bing Search API 는 강력한 성능을 제공하지만, 사용량에 따른 비용이 발생하고 API 키 발급 및 관리의 복잡성이 존재합니다. 이에 반해 덕덕고(DuckDuckGo)는 개발자 친화적인 무료 API 를 제공하며, 무엇보다 '사용자 프라이버시 보호'를 핵심 가치로 내세우고 있습니다.

덕덕고는 사용자의 검색 기록이나 개인 정보를 수집하지 않기 때문에, 민감한 정보를 다룰 수 있는 챗봇 애플리케이션에서 윤리적인 선택지가 될 수 있습니다. 또한 duckduckgo-search 파이썬 패키지와 랭체인(DuckDuckGoSearchResults 도구를 통해 손쉽게 통합이 가능하므로, 프로토타이핑 및 연구 목적의 개발에 최적화되어 있습니다.

2.2 LangChain 을 활용한 검색 도구 구현 및 데이터 구조 분석

2.2.1 기본 검색 구현과 데이터 반환 형식

랭체인(DuckDuckGoSearchResults 클래스는 검색 쿼리를 받아 검색 결과를 반환하는 래퍼(Wrapper) 역할을 수행합니다. 가장 기본적인 사용법은 다음과 같습니다.

```
from langchain_community.tools import DuckDuckGoSearchResults
```

```
# 검색 결과 구분자 설정
```

```
search = DuckDuckGoSearchResults(results_separator=';\\n')
```

```
docs = search.invoke("최근 로제가 발표한 신곡은 무엇인가요?")
```

이 코드를 실행하면, 검색 엔진은 사용자의 쿼리를 처리하여 관련성 높은 웹 페이지들의 리스트를 반환합니다. 반환되는 데이터는 단순한 문자열이 아니라, 각 검색 결과 항목이 snippet(요약), title(제목), link(URL) 등의 메타데이터를 포함하는 구조화된 텍스트 형식입니다. 이때 results_separator 파라미터는 여러 개의 검색 결과를 구분하는 데 사용되며, 이를 통해 후처리 과정에서 각 결과를 개별적으로 파싱하기 용이해 집니다.

2.2.2 DuckDuckGoSearchAPIWrapper 를 통한 고급 검색 제어

검색 품질 제어: DuckDuckGoSearchAPIWrapper를 활용한 정밀 타겟팅

핵심: 단순 키워드 검색은 '과일 애플'과 'IT 기업 애플'을 혼동하거나 오래된 뉴스를 가져올 수 있습니다. `DuckDuckGoSearchAPIWrapper`를 통해 검색을 세밀하게 제어해야 합니다.

주요 파라미터



`region="kr-kr"`: 한국어 사용자 대상 서비스의 정확도 향상



`time="w"` (week): 최신성이 중요한 뉴스, 시장 동향 검색



`max\_results=5`: LLM의 컨텍스트 윈도우에 맞춰 토큰 사용량 최적화



`source="news"`: 신뢰성 있는 언론사 기사로 검색 소스 한정

```
# Annotated Python code snippet
from langchain_community.utilities import DuckDuckGoSearchAPIWrapper

# 래퍼 설정: 한국 지역, 최근 일주일, 뉴스 소스 한정
wrapper = DuckDuckGoSearchAPIWrapper(
    region="kr-kr",  # 한국어 서비스 타겟팅
    time="w",        # 최신 1주일 정보만 검색
    max_results=5    # 토큰 비용 및 컨텍스트 최적화
)

search = DuckDuckGoSearchResults(api_wrapper=wrapper, source="news")
```

결과

이 설정을 통해 '로제 신곡' 검색 시, 1년 전 앨범이 아닌 최근 1주일 내의 뉴스 기사만 반환되어 환각을 극적으로 감소시킴.

단순한 키워드 검색만으로는 챗봇의 답변 품질을 보장하기 어렵습니다. 예를 들어, "애플"을 검색했을 때 과일 애플과 IT 기업 애플이 혼재될 수 있으며, 1년 전의 뉴스가 최신 뉴스보다 상위에 노출될 수도 있습니다. 이를 제어하기 위해 DuckDuckGoSearchAPIWrapper를 사용하여 검색 파라미터를 세밀하게 조정해야 합니다.

주요 파라미터 상세 분석:

파라미터 (Parameter)	기능 및 기술적 의미	활용 시나리오
region	검색 결과를 특정 국가/언어권으로 제한합니다. (예: "kr-kr")	한국어 사용자 대상 서비스에서 국내 이슈나 한국어 문서를 우선적으로 확보할 때 필수적입니다.
time	검색 대상의 시간 범위를 설정합니다. ("d": 1 일, "w": 1 주, "m": 1 달)	급변하는 뉴스나 주식 시장 동향, 재난 상황 등 최신성이 생명인 정보를 다룰 때 사용합니다.
max_results	반환할 검색 결과의 최대 개수를 지정합니다.	LLM 의 컨텍스트 윈도우(Context Window) 제한을 고려하여 토큰 사용량을 최적화할 때 조정합니다.
source	검색 소스의 유형을 지정합니다. (예: "news", "text")	일반 블로그나 커뮤니티 글을 배제하고, 신뢰성 있는 언론사의 기사만을 검색하고 싶을 때 유효합니다.

구현 코드 예시:

```
from langchain_community.utilities import DuckDuckGoSearchAPIWrapper

# 래퍼 설정: 한국 지역, 최근 일주일, 뉴스 소스
wrapper = DuckDuckGoSearchAPIWrapper(region="kr-kr", time="w", max_results=5)

# 검색 도구에 래퍼 주입
search = DuckDuckGoSearchResults(
    api_wrapper=wrapper,
    source="news",
    results_separator=';\n'
)
```

이와 같이 래퍼를 구성하면, "로제 신곡"을 검색했을 때 1 년 전의 앨범 정보가 아닌, 최근 일주일

내에 발행된 뉴스 기사들만이 검색 결과로 반환되어 환각 현상을 획기적으로 줄일 수 있습니다.¹

2.3 WebBaseLoader 와 BeautifulSoup 을 이용한 심층 스크래핑 전략

한계점 (The Limitation): 스니펫(Snippet)은 깊이가 없다

검색 결과의 요약문(snippet)은 제목과 몇 줄의 텍스트에 불과합니다.

이는 피상적인 정보 확인에는 유용하지만, 심층 분석 보고서나 상세한 답변을 생성하기에는 정보량이 절대적으로 부족합니다.

LLM은 전체 문맥 없이는 nuanced한 답변을 생성할 수 없습니다.



다음 단계로의 전환 (The Next-Level Solution)

단순히 검색 결과를 나열하는 것을 넘어, 각 링크의 원문 콘텐츠 전체를 확보해야 합니다.

이를 위해 웹 페이지의 본문을 직접 읽어오는 '심층 스크래핑' 기술이 필요합니다.



검색 결과로 얻은 스니펫(Snippet)은 정보량이 제한적입니다. 챗봇이 심도 있는 답변을 하기 위해서는 해당 링크(URL)를 타고 들어가 웹 페이지의 본문 전체를 읽어오는 과정이 필요합니다.

2.3.1 WebBaseLoader의 아키텍처와 비동기 처리



랭체인 WebBaseLoader는 urllib과 requests 라이브러리를 기반으로 웹 페이지의 HTML을 가져오는 도구입니다. 그러나 동기(Synchronous) 방식으로 수십 개의 URL을 순차적으로 방문하면 응답 시간이 매우 길어집니다. 이를 해결하기 위해 `alazy_load()` 메서드를 활용한 비동기(Asynchronous) 처리가 필수적입니다.⁴

비동기 처리의 이점:

- **속도 향상:** I/O 바운드 작업인 네트워크 요청을 병렬로 처리하여 전체 로딩 시간을 단축합니다.
- **리소스 효율성:** 블로킹 없이 자원을 효율적으로 사용하여 서버 부하를 줄입니다.

```
from langchain_community.document_loaders import WebBaseLoader
```

```
# 비동기 로더 인스턴스화
```

```
loader = WebBaseLoader(
```

```
    web_paths=links, # 검색된 URL 리스트
```

```
    bs_get_text_kwargs={"strip": True} # 텍스트 추출 시 공백 제거 옵션
```

```
)
```

```
# 비동기 제너레이터를 리스트로 변환
page_contents =
async for doc in loader.alazy_load():
    page_contents.append(doc)
```

위 코드는 검색된 모든 링크의 내용을 비동기적으로 가져와 page_contents 리스트에 저장합니다. 이때 bs_get_text_kwargs 는 내부적으로 사용되는 BeautifulSoup 의 get_text() 메서드에 전달될 인자들을 설정하는 것으로, 불필요한 공백을 제거하여 데이터의 밀도를 높입니다.

2.3.2 BeautifulSoup 을 이용한 지능형 DOM 파싱 및 노이즈 제거

웹 페이지는 기사 본문 외에도 광고, 내비게이션 메뉴, 푸터, 댓글 등 챗봇에게 불필요한 정보(Noise)를 다수 포함하고 있습니다. 이러한 노이즈가 LLM 에 입력되면 토큰을 낭비할 뿐만 아니라, 챗봇이 광고 문구를 기사 내용으로 오인하는 등의 오류를 범할 수 있습니다. 따라서 BeautifulSoup 라이브러리를 사용하여 HTML DOM(Document Object Model) 트리에서 핵심 콘텐츠만을 정밀하게 추출해야 합니다.

언론사별 DOM 구조 분석 및 대응:

대부분의 뉴스 사이트는 본문을 특정 태그(예: <article>, <div class="content">) 안에 배치합니다. 그러나 사이트마다 사용하는 클래스 명이나 ID 가 다르기 때문에, 범용적인 파싱 로직을 구현하는 것이 기술적 난제입니다.

고급 파싱 함수 구현 전략:

```
import requests
from bs4 import BeautifulSoup

def get_article_text(url):
    try:
        # 헤더 설정으로 봇 차단 우회 (User-Agent 설정)
        headers = {'User-Agent': 'Mozilla/5.0...'}
        response = requests.get(url, headers=headers)
        soup = BeautifulSoup(response.content, 'html.parser')
        # 1. 의미론적 태그(Semantic Tag) 우선 검색
```

```

article = soup.find('article')
# 2. 특정 클래스/ID 기반 검색 (휴리스틱 적용)
if not article:
    article = soup.find('div', class_='story-news article') # 연합뉴스 예시
if not article:
    article = soup.find('div', id='articleBody') # 일반적인 뉴스 사이트
# 3. 텍스트 추출 및 정제
if article:
    return article.get_text(strip=True)
else:
    return "본문 영역을 특정할 수 없습니다."
except Exception as e:
    return f"스크래핑 오류: {e}"

```

이 함수는 다단계의 검증 로직을 통해 본문 추출의 성공률을 높입니다. requests 라이브러리로 HTML 을 가져오고, BeautifulSoup 으로 파싱 트리를 생성한 후, find() 메서드를 통해 원하는 태그를 탐색합니다.¹ 특히 get_text(strip=True) 메서드는 태그 내의 텍스트만 깔끔하게 추출하여 LLM 이 처리하기 최적화된 상태로 만듭니다.

스크래핑 엔지니어링: 속도와 품질을 모두 잡는 기술

Part 1: 속도 - 비동기(Asynchronous) 로딩

문제: 수십 개의 URL을 순차적으로 로드하면 I/O 대기로 인해 응답 시간이 급격히 길어집니다.

해결: `alazy_load()` 메서드를 사용하여 모든 네트워크 요청을 병렬로 처리, 전체 로딩 시간을 단축합니다.

```

# Annotated Python code snippet
loader = WebBaseLoader(web_paths=links)
# alazy_load()는 비동기 제너레이터를 반환하여 병렬 처리를 가능하게 함
page_contents = [doc async for doc in loader.alazy_load()]

```

Part 2: 품질 - 지능형 DOM 파싱

문제: 웹 페이지는 본문 외 광고, 메뉴, 댓글 등 노이즈가 많아 LLM을 오염시킬 수 있습니다.

해결: BeautifulSoup을 이용해 다단계 검증 로직으로 본문만 정확히 추출합니다.

```

# Annotated Python code snippet
def get_article_text(url):
    # ... requests.get ...
    soup = BeautifulSoup(response.content, 'html.parser')
    # 1. 시맨틱 태그 <article> 우선 검색
    article = soup.find('article')
    # 2. 사이트별 특정 클래스/ID 기반의 휴리스틱 적용
    if not article:
        article = soup.find('div', class_='story-news article')
    # ...
    return article.get_text(strip=True) if article else ""

```

3. 차세대 리서치 에이전트: Tavily API 를 활용한 자동화된 저널리즘

3.1 LLM 전용 검색 엔진 Tavily 의 등장이 갖는 의미

Level 3. 우아한 도약: LLM을 위해 태어난 검색 엔진, Tavily

패러다임 전환

- DuckDuckGo나 Google은 **인간**을 위해 설계되었습니다.
- Tavily는 처음부터 **LLM 에이전트**를 위해 설계된 검색 API입니다.

Tavily의 혁신

- 복잡한 `검색 -> URL 추출 -> 비동기 스크래핑 -> HTML 파싱 -> 텍스트 정제` 파이프라인을 **단 한 번의 API 호출**로 통합합니다.
- LLM이 즉시 사용할 수 있는 깨끗한 텍스트(Clean Text) 또는 마크다운 형식으로 결과를 반환합니다.

아키텍처 다이어그램 v4



Tavily가 게임 체인저인 이유

개발자가 직접 구현해야 했던 복잡한 Search-Scrape-Clean 파이프라인 전체를 하나의 API 호출로 대체합니다. 이는 엄청난 개발 시간과 노력을 절약하며, 시스템의 안정성을 높입니다.

DuckDuckGo 나 Google Search API 는 인간 사용자를 위해 설계된 검색 엔진입니다. 따라서 검색 결과에는 광고가 포함되어 있고, 페이지 구조가 복잡하며, 봇 접근을 차단하는 캡차(CAPTCHA) 등이 존재합니다. 반면 **Tavily** 는 기획 단계부터 LLM 에이전트를 위해 설계된 검색 API 입니다. Tavily 는 검색, 스크래핑, 필터링, 요약을 단 한 번의 API 호출로 수행하며, LLM 이 이해하기 쉬운 깨끗한 텍스트(Clean Text)나 마크다운 형식을 반환합니다.¹ 이는 개발자가 복잡한 스크래핑 및 파싱 로직을 직접 구현할 필요성을 제거하여 개발 생산성을 비약적으로 높여줍니다.

3.2 Tavily API의 아키텍처 및 핵심 기능 분석

단 한 번의 API 호출로 리서치 자동화하기

Tavily 핵심 기능

`search\_depth`

`basic`: 빠른 응답 속도

`advanced`: 더 깊이 있는 고품질 정보 탐색 (리서치에 필수)

`include\_raw\_content`

`True`로 설정 시, 검색된 각 페이지의 본문

전체(raw_content)를 정제된 텍스트로 함께 반환.

이는 `WebBaseLoader` + `BeautifulSoup` 프로세스를 완벽하게 대체합니다.

```
# Annotated Python code snippet
from tavily import TavilyClient

client = TavilyClient(api_key="tvly-...")
response = client.search(
    query="2025년 한국 경제 전망",
    search_depth="advanced",
    include_raw_content=True,
    max_results=5
)
# response['results']의 각 항목에 'raw_content' 필드가 포함됨
```

심층 리서치 모드

원문 콘텐츠 포함

활용

`raw\_content`에 담긴 고품질 데이터를 LLM 프롬프트에 직접 주입하여, 단순 요약을 넘어 전문적인 심층 분석 리포트 생성이 가능해집니다.

Tavily는 단순 검색을 넘어 '검색 깊이(Search Depth)'와 '콘텐츠 정제(Raw Content Inclusion)' 기능을 제공합니다.

- **Search Depth (basic vs advanced):** basic은 빠른 응답 속도를 제공하지만, advanced는 더 깊이 있는 검색을 수행하여 고품질의 정보를 찾아냅니다.⁸
- **Raw Content Inclusion:** 검색된 페이지의 전체 텍스트를 파싱하여 제공합니다. 이는 앞서 DuckDuckGo 섹션에서 구현했던 WebBaseLoader + BeautifulSoup 과정을 API 내부에서 자동으로 처리해주는 것과 같습니다.

3.2.1 TavilyClient 구현 및 파라미터 최적화

Tavily를 랭체인에서 사용하는 방법은 크게 TavilySearchResults 도구를 사용하는 방법과 TavilyClient를 직접 사용하는 방법이 있습니다. 심층 리서치 리포트 작성을 위해서는 원본 콘텐츠를 확보할 수 있는 TavilyClient가 유리합니다.

```
from tavily import TavilyClient
```

```
# 클라이언트 초기화
```

```

client = TavilyClient(api_key="tvly-...")

# 심층 검색 수행
response = client.search(
    query="2025 년 한국 경제 전망",
    search_depth="advanced",      # 심층 검색 모드 활성화
    include_raw_content=True,     # 본문 원본 데이터 포함
    max_results=5                 # 상위 5 개 결과 확보
)

```

위 코드를 실행하면, 각 결과 항목에 `raw_content` 필드가 포함되어 돌아옵니다. 이 필드에는 HTML 태그나 스크립트가 제거된 순수 텍스트가 담겨 있어, 이를 그대로 LLM 의 프롬프트에 입력할 수 있습니다.¹

3.3 프롬프트 엔지니어링을 통한 AI 기자 에이전트 구축

확보된 고품질의 데이터를 바탕으로, LLM 에게 전문적인 기사를 작성하도록 지시하는 프롬프트 엔지니어링 단계입니다. 여기서 핵심은 LLM 에게 명확한 '페르소나(Persona)'를 부여하고, 출력 형식을 강제하는 것입니다.

프롬프트 설계 전략:

1. **Role Definition:** "당신은 전문적인 신문 기자 AI 입니다."라는 시스템 메시지를 통해 모델의 톤앤매너(Tone and Manner)를 설정합니다.
2. **Context Injection:** Tavily 로 수집한 `raw_content` 데이터를 프롬프트 내에 문맥으로 제공합니다.
3. **Task Specification:** "MLA 표준을 준수하는 마크다운 문법을 사용하라", "출처를 명시하라" 등의 구체적인 지시 사항을 포함합니다.¹

코드 구현:

```

# 프롬프트 구성
prompt = [
    "role": "system",
    "content": "당신은 주어진 정보를 바탕으로 객관적이고 체계적으로 기사를 작성하는 전문

```

기자입니다."

```
}, {  
    "role": "user",  
    "content": f'정보: ""{content}" "WnWn 위의 정보를 사용하여 "현대자동차 미국 시장  
2025 년 전망"에 대한 심층 리포트를 작성하세요. 반드시 출처를 명기하십시오.'  
}]
```

LLM 호출 및 결과 생성

```
report = llm.invoke(prompt).content  
print(report)
```

이 과정을 통해 생성된 리포트는 단순한 요약문이 아니라, 검색된 통계 자료(예: 판매량 증가율, 시장 점유율 등)와 전문가의 견해를 인용한 고품질의 분석 기사가 됩니다. 이는 RAG 시스템이 단순한 챗봇을 넘어 지식 생산 도구로 활용될 수 있음을 시사합니다.

4. 멀티미디어 데이터의 텍스트화: 유튜브 인텔리전스 파이프라인

4.1 텍스트 중심 검색의 한계와 비디오 데이터의 중요성

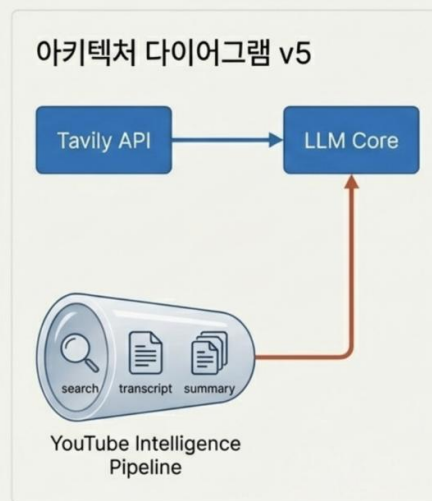
세계관 확장: 텍스트를 넘어 영상으로, 왜 YouTube인가?

텍스트 중심 검색의 한계

- 현대 정보의 상당 부분은 비디오 형식, 특히 YouTube를 통해 유통됩니다.
- 뉴스 속보, 제품 리뷰, 기술 강의 등은 텍스트 기사보다 영상으로 먼저 공개되는 경우가 많습니다.
- 텍스트 검색에만 의존하는 챗봇은 정보의 최신성과 다양성 측면에서 뒤처질 수밖에 없습니다.

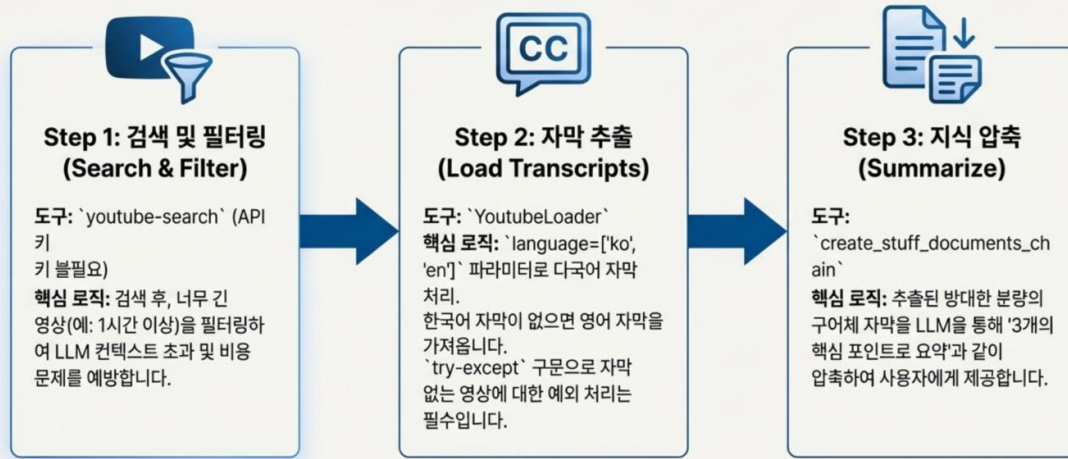
해결 전략: 비디오의 텍스트화

- LLM은 비디오를 직접 '시청'할 수 없습니다.
- 따라서 비디오의 음성을 텍스트로 변환한 **자막(Transcript)**을 추출하여 LLM이 이해할 수 있는 형태로 변환하는 우회 전략이 필요합니다.



현대 정보의 상당 부분은 텍스트가 아닌 비디오 형식, 특히 유튜브(YouTube)를 통해 유통됩니다. 뉴스 속보, 제품 리뷰, 기술 강의 등은 텍스트보다 영상으로 먼저 공개되는 경우가 많습니다. 따라서 텍스트 기반 검색에만 의존하는 챗봇은 정보의 최신성과 다양성 측면에서 한계를 가질 수밖에 없습니다. 유튜브 데이터를 검색하고 이해하는 능력은 챗봇의 지식 범위를 획기적으로 확장시킵니다. 그러나 LLM은 비디오 파일을 직접 '시청'할 수 없으므로, 비디오의 오디오를 텍스트로 변환한 '자막(Transcript)'을 활용하는 우회 전략이 필요합니다.

YouTube 인텔리전스 파이프라인 구축: 3단계 프로세스



4.2 Youtube Search 패키지를 이용한 영상 메타데이터 확보

먼저 사용자의 질문과 관련된 영상을 찾아야 합니다. 랭체인인 도구 대신 youtube-search 파이썬 패키지를 사용하면 API 키 없이도 검색 결과를 가져올 수 있어 접근성이 좋습니다.

검색 및 전처리 로직:

단순히 검색 결과를 가져오는 것을 넘어, 챗봇이 처리 가능한 영상인지 선별하는 과정이 중요합니다. 예를 들어, 10 시간이 넘는 라이브 스트리밍 영상은 자막 데이터가 너무 방대하여 LLM의 컨텍스트 윈도우를 초과하거나 토큰 비용을 급증시킬 수 있습니다.

영상 필터링 알고리즘:

```
from youtube_search import YoutubeSearch
```

```
# 1. 키워드 검색 수행
```

```
results = YoutubeSearch("미국 대선 분석", max_results=10).to_dict()
```

```
# 2. 영상 길이 기반 필터링 (1 시간 미만 영상만 선택)
```

```
filtered_videos =
```

```
for video in results:
```

```

duration = video['duration'] # 예: '10:05', '1:05:20'
time_parts = duration.split(':')

# 시간 단위(시:분:초)가 없거나, 1 시간 미만인 경우만 허용
if len(time_parts) < 3:
    filtered_videos.append(video)

```

이러한 전처리 과정은 시스템의 안정성과 응답 속도를 보장하는 핵심적인 엔지니어링 기법입니다.

4.3 YoutubeLoader 와 자막 추출 기술

선별된 영상의 URL 을 이용하여 자막을 추출합니다. 랭체인(YoutubeLoader)은 내부적으로 youtube-transcript-api 를 사용하여 자막을 가져옵니다.

다국어 자막 처리 전략:

글로벌 정보를 다룰 때, 한국어 자막이 없는 영상이 있을 수 있습니다. 이럴 때는 영어나 기타 언어 자막을 가져오도록 우선순위를 설정해야 합니다. language=['ko', 'en'] 파라미터는 한국어 자막을 먼저 찾고, 없으면 영어 자막을 가져오도록 지시합니다.

```

from langchain_community.document_loaders import YoutubeLoader

```

```

# 영상별 자막 로드
for video in filtered_videos:
    loader = YoutubeLoader.from_youtube_url(
        video['url'],
        language=['ko', 'en'], # 언어 우선순위 설정
        add_video_info=True    # 영상 메타데이터 포함
    )
    try:
        transcript = loader.load()
        video['content'] = transcript
    except Exception as e:
        print(f"자막 추출 실패 ({video['title']}): {e}")

```

이때 자막이 비활성화된 영상의 경우 예외가 발생할 수 있으므로, try-except 구문을 통해 파이프라인이 중단되지 않도록 처리하는 것이 중요합니다.

4.4 요약 체인(Summarization Chain)을 통한 지식 압축

추출된 자막은 구어체이며 분량이 많습니다. 이를 그대로 사용자에게 보여주는 것은 비효율적이므로, LLM 을 통해 핵심 내용만 요약해야 합니다. 랭체인 `create_stuff_documents_chain` 은 문서를 컨텍스트에 '채워 넣는(Stuff)' 방식으로 요약을 수행합니다.

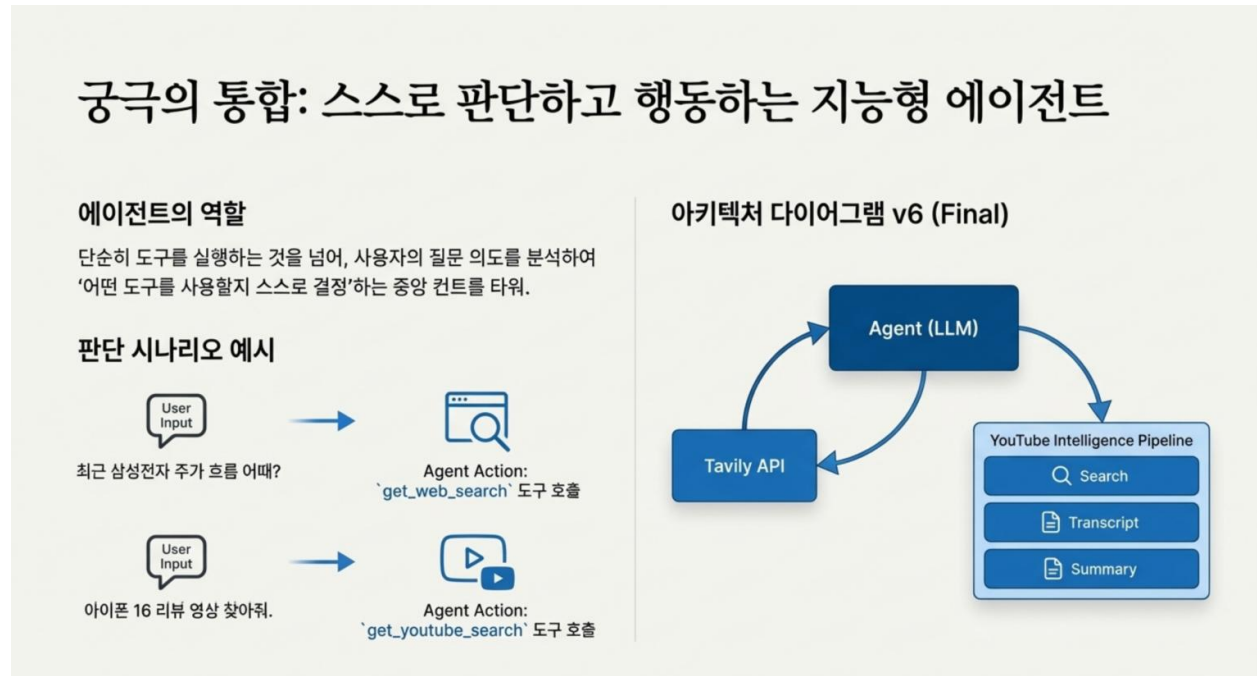
요약 프롬프트 예시:

"다음은 유튜브 영상의 자막입니다. 이 영상의 주요 주장과 결론을 3 개의 핵심 포인트로 요약하여 한국어로 작성해주세요."

이 과정을 거치면 수십 분 분량의 영상이 몇 줄의 텍스트로 압축되어, 사용자가 영상을 직접 보지 않고도 내용을 파악할 수 있게 됩니다.

5. 통합 에이전트 아키텍처 및 Streamlit 구현 전략

5.1 통합 챗봇의 아키텍처: 판단하는 에이전트



이제 웹 검색(DuckDuckGo)과 유튜브 검색(YouTube Search) 기능을 하나의 시스템으로 통합합니다. 이 시스템의 핵심은 LLM 이 사용자의 질문을 분석하여 '어떤 도구를 사용할지 스스로 결정'하는 능력입니다.

시나리오별 에이전트의 판단:

- **Case A:** "지금 뉴욕 시간 몇 시야?" -> `get_current_time` 도구 호출.
- **Case B:** "최근 삼성전자 주가 흐름 어때?" -> `get_web_search` 도구 호출.
- **Case C:** "아이폰 16 리뷰 영상 찾아줘." -> `get_youtube_search` 도구 호출.

5.2 @tool 데코레이터와 함수 바인딩(Function Binding)

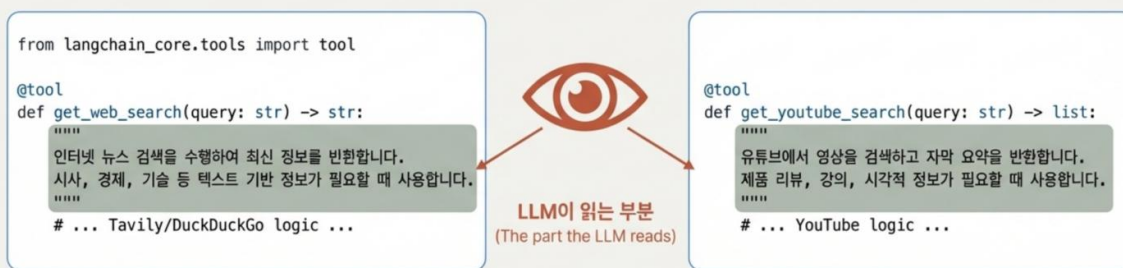
에이전트는 어떻게 생각하는가: Docstring의 결정적 역할

핵심 메커니즘

LLM은 코드 자체를 실행하는 것이 아니라, 함수의 **설명문(Docstring)**을 읽고 그 도구의 목적과 사용법을 이해합니다.

구현

1. **@tool** 데코레이터를 사용하여 파이썬 함수를 LLM이 사용할 수 있는 '도구'로 등록합니다.
2. LLM이 명확히 이해할 수 있도록 Docstring에 도구의 기능, 파라미터, 사용 시나리오를 상세히 기술합니다.



LLM 이 파이썬 함수를 도구로 인식하게 하려면 **@tool** 데코레이터를 사용해야 합니다. 이때 가장 중요한 것은 함수의 ****Docstring(설명문)****입니다. LLM 은 이 설명문을 읽고 도구의 사용 목적과 파라미터 형식을 이해합니다.

```
from langchain_core.tools import tool
```

```
@tool
```

```
def get_web_search(query: str, search_period: str = "w") -> str:
```

```
    """
```

```
    인터넷 뉴스 검색을 수행하여 최신 정보를 반환합니다.
```

```
    Args:
```

```
        query: 검색어
```

```
        search_period: 검색 기간 ('d': 하루, 'w': 일주, 'm': 한 달)
```

```
    """
```

```
    #... DuckDuckGo 로직...
```

```
    return results
```

```
@tool
def get_youtube_search(query: str) -> list:
    """
    유튜브에서 영상을 검색하고 자막 요약을 반환합니다.
    시각적인 정보나 영상 리뷰가 필요할 때 사용합니다.
    """

    #... YouTube 로직...

    return video_data
```

이후 `llm.bind_tools([get_web_search, get_youtube_search])`를 통해 모델과 도구를 연결합니다.

5.3 Streamlit 을 활용한 대화형 인터페이스 구축

경험의 완성: Streamlit을 활용한 대화형 인터페이스 구축

핵심 구현 요소

세션 상태 (Session State) 관리:

``st.session_state``를 사용하여 이전 대화 기록을 유지, 멀티턴(Multi-turn) 대화를 가능하게 합니다.

에이전트 실행 루프: 사용자 입력을 받아 에이전트를 실행하고, 에이전트의 도구 호출 요청을 감지하여 해당 함수를 실행한 후, 그 결과를 다시 에이전트에게 전달하여 최종 답변을 생성합니다.

멀티미디어 렌더링: 유튜브 검색 결과는 단순 텍스트가 아닌, ``st.image()``로 썸네일을, ``st.link_button()``으로 영상 링크를 제공하여 시각적으로 풍부한 사용자 경험을 제공합니다.

멀티모달 검색 챗봇

아이폰 16 최신 리뷰 영상이란 관련 뉴스 기사 요약해줘.

아이폰 16은 향상된 A18 칩, 새로운 디자인, 그리고 향상된 카메라 시스템을 탑재하여 전반적인 성능과 사용자 경험이 크게 개선되었습니다. 관련 뉴스에서는 이러한 혁신적인 기능들과 시장의 반응을 중점적으로 다루고 있습니다.



iPhone 16 Full Review: The Game Changer is Here!
TechInsider

Watch on YouTube

Streamlit 은 파이썬만으로 데이터 애플리케이션을 만들 수 있는 강력한 프레임워크입니다. 챗봇을 구현할 때 가장 중요한 것은 **세션 상태(Session State)** 관리입니다. 이전 대화 기록(messages)을 유지해야 멀티턴(Multi-turn) 대화가 가능합니다.

Streamlit 구현의 핵심 요소:

1. **세션 초기화:** `st.session_state` 에 메시지 리스트가 없으면 초기화합니다.
2. **사용자 입력 처리:** `st.chat_input()`으로 입력을 받고 화면에 출력합니다.

3. **에이전트 실행:** 사용자 입력을 LLM 에 전달하고, LLM 이 도구 호출을 요청하면 해당 함수를 실행한 뒤 결과를 다시 LLM 에 전달합니다.
4. **멀티미디어 렌더링:** 유튜브 검색 결과의 경우, 단순 텍스트뿐만 아니라 썸네일 이미지와 링크를 `st.image()`, `st.link_button()` 등을 사용하여 시각적으로 풍부하게 표현합니다.

통합 코드 구조 (개념도):

```
import streamlit as st
from langchain_openai import ChatOpenAI

# 1. 모델 및 도구 설정
llm = ChatOpenAI(model="gpt-4o", temperature=0)
llm_with_tools = llm.bind_tools(tools)

# 2. UI 레이아웃
st.title("🔍 멀티모달 검색 챗봇")

# 3. 대화 루프
if prompt := st.chat_input():
    st.session_state.messages.append({"role": "user", "content": prompt})
    # 에이전트 추론
    response = llm_with_tools.invoke(st.session_state.messages)
    # 도구 호출 감지 및 실행 로직
    if response.tool_calls:
        #... 도구 실행 및 결과 반환...

    # 최종 답변 출력
    st.chat_message("assistant").write(final_answer)
```

이러한 구조를 통해 사용자는 텍스트 질문 하나만으로 최신 뉴스 기사와 관련 유튜브 영상을 한 화면에서 동시에 확인하는 입체적인 정보 습득 경험을 할 수 있습니다.

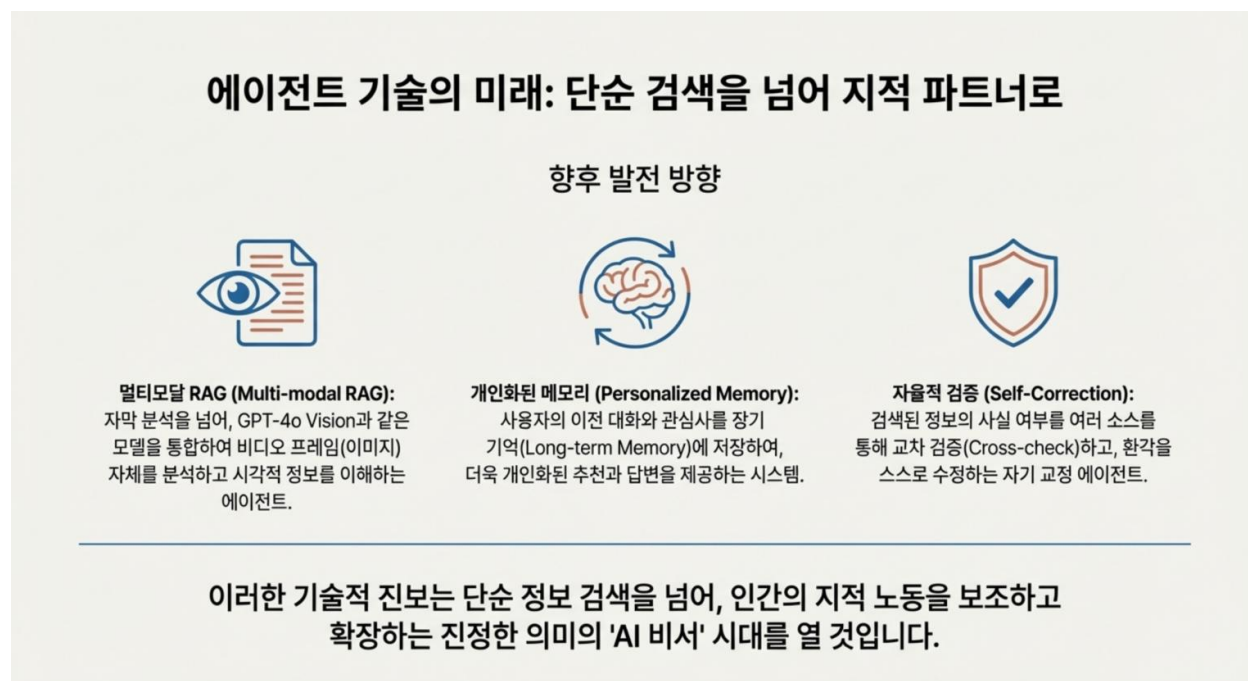
6. 결론 및 향후 전망: 에이전트 기술의 미래

6.1 연구 요약

본 보고서에서는 LLM의 지식 단절 문제를 극복하기 위해 웹 검색과 유튜브 자막 분석 기술을 통합하는 방법을 심도 있게 다루었습니다.

1. **DuckDuckGo**를 통한 실시간 정보 검색과 BeautifulSoup을 이용한 정밀한 데이터 정제 기술을 확보했습니다.
2. **Tavily**라는 LLM 전용 검색 엔진을 통해 복잡한 리서치 업무를 자동화하는 에이전트 구축 가능성을 확인했습니다.
3. **YouTube** 데이터 파이프라인을 구축하여 텍스트를 넘어선 멀티미디어 정보 처리 능력을 갖추었습니다.
4. **Streamlit**을 통해 이 모든 기술을 사용자가 쉽게 접근할 수 있는 애플리케이션 형태로 구현했습니다.

6.2 기술적 제언 및 발전 방향



향후 이 시스템은 다음과 같은 방향으로 발전할 수 있습니다.

- **멀티모달 RAG:** 단순히 자막만 분석하는 것을 넘어, 비디오의 프레임(이미지)을 직접 분석하는 Vision Model(예: GPT-4o Vision)을 통합하여 시각적 정보까지 이해하는 에이전트.

- **개인화된 메모리:** 사용자의 관심사와 이전 검색 기록을 장기 기억(Long-term Memory)에 저장하여, 더욱 개인화된 추천과 답변을 제공하는 시스템.
- **자율적 검증:** 검색된 정보의 사실 여부를 교차 검증(Cross-check)하여 환각을 스스로 수정하는 자기 교정(Self-Correction) 에이전트.

이러한 기술적 진보는 단순한 정보 검색을 넘어, 인간의 지적 노동을 보조하고 확장하는 진정한 의미의 'AI 비서' 시대를 앞당길 것입니다.